

This one covers goroutines — Go's concurrency primitive. Given you're already running goroutine workers in your email pipeline, this is the foundation under that. Here's the full picture.

The baseline: sequential, blocking

Start with a task function and a loop:

```
func task(id int) {
    fmt.Println("doing task", id)
}

func main() {
    for i := 0; i <= 10; i++ {
        task(i)
    }
}
```

Output: `doing task 0`, `1`, `2`, ... `10` — in exact order. This is **blocking**: `task(0)` must fully finish before `task(1)` starts. The loop waits on each call. Normal synchronous execution.

Adding `go` makes it concurrent

```
func main() {
    for i := 0; i <= 10; i++ {
        go task(i)
    }
}
```

Putting `go` before the call launches it as a **goroutine** — a lightweight thread managed by the Go runtime, not the OS. The call no longer blocks. The loop fires off all 11 goroutines and immediately moves on.

But if you run this as-is, you see **nothing**. No output at all. That's the critical lesson in this transcript, and it's worth understanding exactly why.

Why nothing prints: main is also a goroutine

`main` itself runs in a goroutine (the "main goroutine"). When you write `go task(i)`, the runtime does **not** run it right then. It hands the goroutine to Go's **scheduler**, which queues it and runs it

whenever it gets a slot. So `go task(i)` returns instantly — it's a non-blocking scheduling operation, not the actual work.

So the loop schedules 11 goroutines in microseconds, then `main` reaches the end of its body. There's nothing left to execute, so **main exits**. And the rule is: when the main goroutine exits, the program exits immediately, and **every pending goroutine is killed** — whether or not it ran. They never got a chance to print.

This is the core mental model: goroutines don't keep the program alive. Main does.

The Sleep "fix" (and why it's a hack)

To see them run, the transcript stalls main so it doesn't exit first:

```
import "time"

func main() {
    for i := 0; i <= 10; i++ {
        go task(i)
    }
    time.Sleep(2 * time.Second)
}
```

Now main sleeps for 2 seconds, giving the scheduler time to run all the goroutines before exit. And now you see output — but the order is scrambled: `doing task 10, 7, 8, 9, 0, 2, 6, 5...`

That scrambling **is** concurrency. The goroutines aren't serialized anymore; the scheduler interleaves them and they finish in whatever order they happen to get scheduled. There's no ordering guarantee. The sequential version was ordered precisely because it was blocking; the concurrent version trades ordering for speed.

The transcript is explicit that `time.Sleep` is a **hack** — you never do this in real code. You can't know how long the work takes, so you'd either sleep too long (slow) or too short (cut goroutines off). The proper tool is `sync.WaitGroup` (or channels), which the course covers later. That's exactly the mechanism that should be in your email pipeline rather than arbitrary sleeps.

Concurrent vs parallel — not the same thing

The transcript makes a distinction that matters. Whether those goroutines run *truly in parallel* (literally at the same instant on different CPU cores) or just *concurrently* (interleaved on one core via context switching) depends on your hardware:

- **One core:** the runtime rapidly switches between goroutines — context switching. They make progress "together" but only one runs at any given instant. That's concurrency.
- **Multiple cores:** the scheduler can place goroutines on different cores and they execute genuinely simultaneously. That's parallelism.

Either way it's fast, because nothing blocks. You write the same `go` code; the runtime decides how to physically execute it based on available cores.

Anonymous goroutines and the closure trap

You don't need a named function. You can launch an inline anonymous function:

```
go func() {  
    fmt.Println(i)  
}()
```

Note the trailing `()` — you're defining *and* immediately invoking the function, with `go` in front. The function body can reach `i` from the enclosing loop because of **closures** — an inner function captures variables from its surrounding scope.

Here's where the transcript gives a best-practice that's actually fixing a famous Go bug. Instead of letting the closure *capture* the loop variable, **pass it as an argument**:

```
go func(id int) {  
    fmt.Println(id)  
}(i)
```

Why this matters: when a closure captures `i` by reference, all the goroutines share the *same* variable. By the time they actually run (remember — scheduled, not immediate), the loop may have already advanced or finished, so several goroutines can read the same final value of `i`. Passing `i` as an argument copies its value *at launch time*, giving each goroutine its own correct, frozen copy. This is the safe pattern.

(Worth knowing: Go 1.22 changed loop semantics so each iteration gets a fresh `i`, which defuses the classic version of this bug. But the pass-as-argument habit is still the clearest, most portable practice — it works regardless of Go version and makes the data flow explicit.)

What this buys you

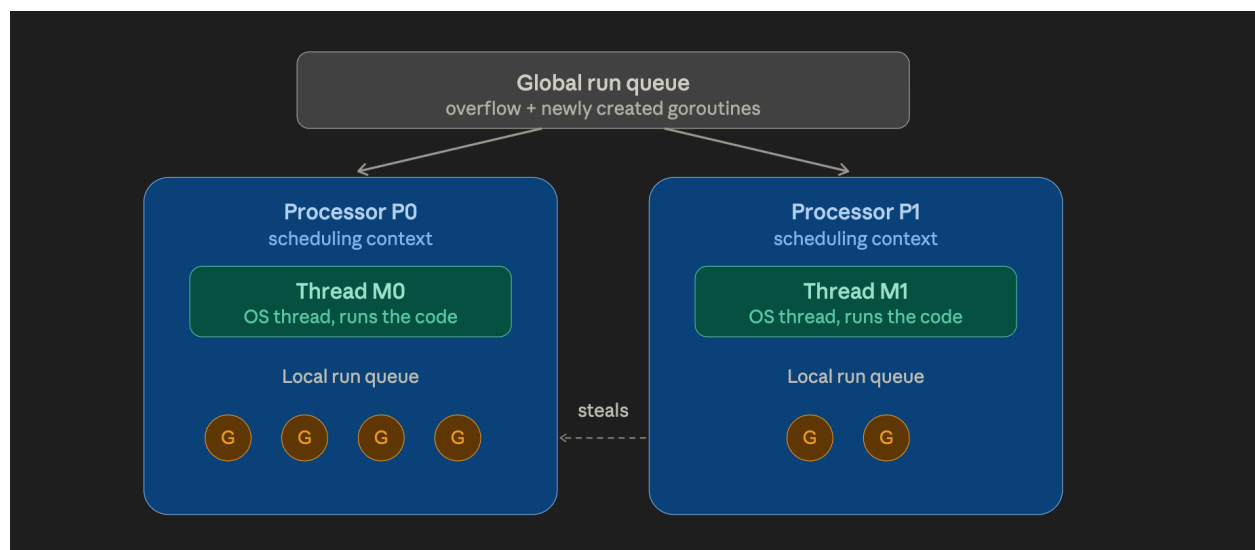
The whole point: `go` is a one-keyword cost to move work off the blocking path. The transcript names the practical uses directly — background workers, CPU-intensive jobs run concurrently,

anything you don't want serializing your main flow. That's exactly the shape of a rate-limited email-send worker pool: many goroutines pulling jobs and hitting SES concurrently instead of one-at-a-time.

The two takeaways to carry forward: goroutines are *scheduled*, *not immediately run*, and the program only lives as long as main — so you need a real synchronization mechanism (WaitGroup/channels), not Sleep, to wait for them properly.

Good question — this is the part that makes goroutines actually interesting. The `go` keyword is trivial; what's underneath is the Go runtime scheduler, built on a model called **G-M-P**. Let me lay out the architecture and then how it handles the hard cases (blocking, stack growth, fairness).

Let me give you a diagram of the core model first. The `go` keyword hands work to a runtime scheduler that maps many goroutines onto a small number of OS threads. The whole thing runs on a model with three entities — **G**, **M**, and **P** — usually called the G-M-P (or GMP) scheduler. Here's how they fit together.## The three entities



G — goroutine. This is the unit of work you create with `go`. A G is a small struct: it holds the goroutine's stack, its instruction pointer (where it's paused), and its scheduling state (runnable, running, waiting, etc.). It is *not* a thread — it's just data describing a job. Creating one is cheap (a few hundred bytes plus a tiny stack), which is why you can spin up hundreds of thousands.

M — machine. This is an actual OS thread — the only thing that can physically execute code on a CPU. The OS schedules M's onto cores; Go schedules G's onto M's. M's are expensive (full kernel threads), so Go keeps the count low and reuses them.

P — processor. This is the piece that makes the whole thing work. A P is a *logical processor* — a scheduling context that owns a **local run queue** of runnable goroutines. The number of P's is fixed at startup by `GOMAXPROCS` (defaults to the number of CPU cores).

The binding rule is the key insight: **an M can only run goroutines if it holds a P.** P is the permission slip. Since there are exactly `GOMAXPROCS` P's, at most that many goroutines run truly in parallel at any instant — no matter how many goroutines or threads exist. That's how Go caps real parallelism to your core count while still juggling thousands of goroutines.

So the runtime relationship is: M (the engine) picks up a P (the context), and the P feeds it G's (the work) from its local queue, one at a time.

How scheduling actually runs

When you write `go task(i)`, the runtime allocates a G and pushes it onto the **local run queue** of the current P. It does not run immediately — that's the "scheduled, not run" point from the last lesson, now made concrete.

Each M, while holding a P, loops: grab the next G from the P's local queue, run it until it yields, repeat. When the local queue empties, the P doesn't just sit idle — it goes looking for work in this order:

1. Its own local run queue (fast, no locking — it's the P's private deque).
2. The **global run queue** (shared, needs a lock — holds overflow and newly spawned goroutines that didn't fit locally).
3. **Work stealing** — it picks another P at random and steals half of that P's local queue.

That work-stealing step (the dashed arrow in the diagram) is what keeps all cores busy. If P0 burns through its goroutines while P1 has a backlog, P0 steals half of P1's queue rather than going idle. This load-balances automatically without a central dispatcher.

There's also a periodic check of the global queue (roughly every 61 scheduler ticks) so goroutines parked there don't starve while P's keep recycling their local queues.

The hard part: blocking

A naive "many goroutines on few threads" design dies the moment one goroutine makes a blocking call — it would block the whole OS thread and stall every other goroutine queued behind it. Go handles three kinds of blocking differently, and this is where the architecture earns its keep.

Blocking syscall (file read, etc.). The M is about to block in the kernel, so the runtime **detaches the P from that M** ("handoff"). The P, plus its queue of waiting goroutines, gets handed to another M (a spare, or a freshly created one) so those goroutines keep running on a

different thread. The original M stays parked in the syscall with just its one blocked G. When the syscall returns, that M tries to reacquire a P; if none is free, its G goes back on a run queue and the M parks for reuse. Net effect: one blocking syscall stalls one goroutine, not the program.

Network I/O. This is special-cased and never blocks an M at all. Go has an **integrated network poller** built on `epoll` (Linux) / `kqueue` (BSD/macOS) / IOCP (Windows). When a goroutine does a network read that isn't ready, the runtime parks *just that goroutine*, marks the socket with the poller, and the M immediately moves on to the next G. When the OS signals the socket is ready, the poller wakes the goroutine and puts it back on a run queue. This is exactly why Go can hold tens of thousands of concurrent connections cheaply — each one is a parked goroutine costing a few KB, not a blocked thread. (Directly relevant to your email pipeline: thousands of in-flight SES HTTP calls are cheap for the same reason.)

Channel / mutex blocking. Pure runtime-internal. A goroutine blocked on a channel send/receive or a lock is just **parked** — moved off the M into a wait list associated with that channel/mutex. The M grabs the next runnable G. When another goroutine sends on the channel (or releases the lock), the parked G is marked runnable and re-queued. No threads involved; this is sub-microsecond.

Preemption — so one goroutine can't hog a core

Originally Go's scheduler was **cooperative**: a goroutine only yielded at function-call boundaries (the compiler inserts safe-points there). A tight loop with no function calls could monopolize its M forever and starve everything else on that P.

Since Go 1.14 the scheduler does **asynchronous preemption**: a background monitor thread (the `sysmon`) notices a goroutine running too long (~10ms), and sends the M a signal. The signal handler stops the goroutine at a safe point, parks it, and lets the P pick another G. So a CPU-bound loop now gets fairly time-sliced like any other goroutine.

`sysmon` is also what retakes P's stuck in long syscalls and kicks the network poller — it's the runtime's watchdog running outside the normal scheduling loop.

Goroutine stacks — why they're cheap

An OS thread reserves a large fixed stack (often 1–8 MB). If goroutines did that, hundreds of thousands of them would exhaust memory instantly. Instead each goroutine starts with a tiny **growable stack** (currently 8 KB). When a function call would overflow it, the runtime allocates a larger stack, copies the existing frames over, fixes up pointers, and resumes — **contiguous stack growth**. It can also shrink stacks during GC. This dynamic sizing is what makes a goroutine cost a few KB instead of a few MB, and it's the other half (with the M/P multiplexing) of why "just launch a goroutine" is a reasonable default in Go.

Putting it together with the last lesson

When you ran `go task(i)` 11 times: 11 G's got pushed onto local run queues. Your `GOMAXPROCS` P's, each driving one M, pulled G's off and ran them — stealing across queues to stay busy — which is why the output order was scrambled. And when `main` (itself a G) finished, the program exited and killed every G still sitting in a queue, because the program's lifetime is tied to the main goroutine, not to the pending ones. The `time.Sleep` hack just kept main's G alive long enough for the scheduler to drain the others — and the proper fix (`sync.WaitGroup`) works by parking main's G on a counter until the worker G's signal completion, which, as you can now see, is just the channel/mutex-style parking mechanism applied to the main goroutine.

If you want, I can go a level deeper on any one piece — the netpoller integration, how `WaitGroup`/channels are implemented on top of this, or how `GOMAXPROCS` tuning affects a throughput-bound workload like your SES sender.